Distributed Systems:

Theory, Architecture & Resilience

Course Overview

This 2-day workshop provides a high-level, topical exploration of Distributed Systems. The requested curriculum covers some of the most complex topics in computer science - from consensus algorithms (Raft/Paxos) to Chaos Engineering. Because building these systems from scratch takes months, this course is heavily focused on **architecture, design patterns, and observability**. Participants will engage in guided theoretical discussions and comparative case studies, observing how real-world tools handle node failures and network partitions.

Note: Examples and architectural case studies will be tailored to Nutanix-relevant cloud infrastructure and virtualization scenarios.

Objectives

- Understand the fundamental tradeoffs of distributed computing (CAP Theorem, PACELC).
- Explore synchronous vs. asynchronous communication (gRPC, Kafka, Pub/Sub).
- Grasp how systems maintain state and consensus (Leader Election, Raft/Paxos).
- Learn how different real-world databases handle network partitions and hardware failures.
- Implement observability strategies to debug microservices at scale.

Duration

2 Days (8 hours/day)

Prerequisites

- Strong software engineering background.
- Familiarity with containerization (Rancher Desktop) and basic networking.

- **MANDATORY:** Environment setup (Rancher Desktop installed, IDE ready) prior to Day 1.

Target Audience

- Software Engineers scaling applications into microservices.
- Systems Architects designing fault-tolerant cloud infrastructure.

Course Outline

Day 1 – Fundamentals, Communication, and Consensus (8 Hours)

Module 1: Fundamentals & Theory

- The 8 Fallacies of Distributed Computing.
- **CAP Theorem & PACELC:** Why distribution is hard and tradeoffs are mandatory.
- **Trainer Example:** A comparative analysis of how a CP system (Etcd/Zookeeper) handles a network partition versus how an AP system (Cassandra) handles the exact same partition.
- Consistency Models: Strong, Eventual, and Causal consistency.

Module 2: Communication & Messaging

- Synchronous vs. Asynchronous architectures.
- **RPC & gRPC:** High-performance service-to-service communication.
- **Message Queues vs. Event Streaming:** Pub/sub, RabbitMQ, and Apache Kafka.
- **Trainer Example:** Hardware failure handling - how RabbitMQ prevents message loss during a node crash versus how Apache Kafka's distributed commit log handles broker failures.
- *Hands-on Lab:* Deploying a basic Pub/Sub architecture and observing message delivery guarantees under simulated network latency.

Module 3: Replication, Consensus & Keeping Data in Sync

- Why keeping data in sync across nodes is the hardest problem in tech.
- **Leader Election:** How clusters agree on a source of truth.
- **Demystifying Raft & Paxos:** A high-level architectural walkthrough.

- **Trainer Example:** Walking through the exact sequence of events when a leader node is physically disconnected (partitioned) from the cluster and how Raft re-establishes consensus.
- *Hands-on Lab:* Standing up a multi-node Etcd or Redis cluster, intentionally killing the leader node, and observing the Raft consensus algorithm elect a new leader.

Module 4: Scalability & Load Distribution

- Scaling out (Horizontal) vs. Scaling up (Vertical).
- **Partitioning & Sharding:** Distributing data across nodes.
- **Consistent Hashing:** How systems rebalance data when nodes are added or removed.
- Load balancing strategies (L4 vs L7).

Day 2 – Resilience, Storage, and Real-World Architectures (8 Hours)

Module 5: Fault Tolerance & Reliability

- Embracing failure: Why systems must be designed to crash.
- **Handling partial failures:** Timeouts, Retries (Exponential Backoff), and Jitter.
- **The Circuit Breaker Pattern:** Preventing cascading failures.
- **Chaos Engineering:** Introduction to intentionally breaking systems.
- **Trainer Example:** Tracing a "Cascading Failure" through a real-world microservice architecture and demonstrating how a Circuit Breaker prevents the entire system from going offline.
- *Hands-on Lab:* Injecting faults and latency into a mock microservice environment to trigger a Circuit Breaker.

Module 6: Distributed Storage & Caching Layers

- **Distributed Databases:** SQL vs. NoSQL tradeoffs in a distributed world.
- **Trainer Example:** How tech giants handle data center failure - comparing Amazon Dynamo's "hinted handoff" eventual consistency vs. Google Spanner's synchronous global replication.
- Read-heavy vs. Write-heavy storage optimizations.
- **Caching Strategies:** Write-through, write-behind, and avoiding cache stampedes.

Module 7: Observability & Debugging at Scale

- Why debugging distributed systems is a nightmare (Logs are not enough).

- **Distributed Tracing:** Implementing OpenTelemetry and Jaeger.
- Centralized logging and visualizing metrics (Prometheus/Grafana).
- *Hands-on Lab:* Using Distributed Tracing to track a single user request across multiple microservices to identify a hidden performance bottleneck.

Module 8: Real-World Architectures & Wrap-up

- **Microservices vs. Monoliths:** The harsh reality of when to use which.
- **Service Meshes:** What Istio and Linkerd actually do (Traffic routing, mTLS).
- **System Design Capstone:** Whiteboarding a highly available, distributed Nutanix-esque service utilizing the patterns learned.
- Course Wrap-up & Next Steps.